GEP 백엔드 설계 기획안 3.0

문서 코드: GEP-005
버전: 3.0
작성일: 2026.02.17
작성자: 노팀장
승인: 조대표님

📋 버전 이력

버전	날짜	주요 변경사항
1.0	2026.02.15	PostgreSQL 풀스택 초안
2.0	2026.02.16	단계적 검증 후 투자 전략 (노션→Supabase→PostgreSQL)
3.0	2026.02.17	통합검증 결과 반영, Phase 1 상세화

3.0 주요 변경:

- Phase 1 기간: “1개월” → **“4주”**
- 노션 API: “직접 호출 + 캐싱” → **“정적 JSON 배포”**
- Rate Limit 대응: “캐싱 전략” → **“JSON 방식으로 완전 회피”**
- Phase 2 전환: “100명 + 30%” → **“300명 + 50%”**
- 문제 수: “240문제 (2개년)” → **“960문제 (23~30회)”**
- Week 0 사전준비 명시 (엑셀→노션 자동화)

---

1. 변경 개요

1.1 핵심 변경사항

기존 1.0 전략:

- Phase 1부터 PostgreSQL 풀스택 구축
- 초기 인프라 투자 필요

2.0 전략:

- 단계적 검증 후 투자
- 노션 → Supabase → PostgreSQL

3.0 확정 전략:

```
검증 → 투자 → 확장
(노션/JSON) → (Supabase) → (PostgreSQL)
```

$0 무료티어 검증 후
```

### 1.2 핵심 철학 (유지)

- **레고블럭:** 모듈 재활용, 각 Phase 독립 동작
- **탑다운:** 100층 건물 기초 = 타 시험 확장 대비
- **품질 우선:** 검증 후 투자
- **빠른 검증:** Phase 1 4주 완성

### 1.3 조대표님 확정 배경

> "기초 공사는 튼튼히, 각 층별로 즉시 오픈"
> = 프론트엔드 우선 + 고객 검증 + 단계적 백엔드 투자

**확정 결정사항 (2026.02.17):**

1. ✅ Phase 1: 4주 (Week 0 사전 3일 포함)
1. ✅ 노션→JSON 주 1회 자동 배포
1. ✅ 23~30회 960문제만 사용
1. ✅ Phase 2 전환: 300명 + 50% 활성도
1. ✅ 도메인: Vercel 기본 사용

---

## 2. Phase별 백엔드 전략

### Phase 1: 노션 + JSON 기반 MVP (4주, $0)

**목표:** 레벨1 무료 서비스로 시장 검증

#### 기술 스택

- **데이터 원본:** 노션 데이터베이스 (읽기 전용)
- **데이터 제공:** 정적 JSON 파일 (주 1회 배포)
- **학습 이력:** 브라우저 localStorage
- **호스팅:** Vercel (무료)
- **비용:** $0

#### 노션 데이터베이스 구조

```
📊 GEP_보험중개사_문제_DB

┌──┐
│ 속성명   │ 타입   │ 설명   │
├──┤
│ 문제ID   │ Title   │ "23-001-보험업법" │

```
│ 회차       │ Number   │ 23~30          │
│ 문번       │ Number   │ 1~120          │
│ 과목       │ Select   │ 1부, 2부, 3부     │
│ 세부과목    │ Select   │ 보험업법, 상법...  │
│ 문제전체    │ Long Text │ 문제+보기 전체     │
│ 정답       │ Number   │ 1, 2, 3, 4      │
│ 공개여부    │ Checkbox │ ✓ (전체 공개)     │
└──────────────────────────────────────────────┘
```

노션→JSON 변환 자동화

Week 0 구축 (Python 스크립트):

```python
# scripts/excel_to_notion.py
import pandas as pd
from notion_client import Client

# 엑셀 로드 (23~30회만)
df = pd.read_excel('ExamBank_Master_Data_V4_5.xlsx')
df_filtered = df[df['회차'] >= 23]  # 960문제

# 노션 연결
notion = Client(auth=NOTION_TOKEN)

# 과목 매핑 테이블
SUBJECT_MAP = {
    '상법': '상법 보험편',
    '자동차': '자동차보험',
    '해상': '해상보험',
    '특종': '특종보험',
    # ... 12개 전체
}

# 각 문제 삽입
for _, row in df_filtered.iterrows():
    notion.pages.create(
        parent={"database_id": DB_ID},
        properties={
            "문제ID": {"title": [{"text": {"content": row['ID']}}]},
            "회차": {"number": row['회차']},
            "문번": {"number": row['문제번호']},
            "과목": {"select": {"name": map_part(row['과목'])}},
            "세부과목": {"select": {"name": SUBJECT_MAP[row['세부과목']]}},
            "문제전체": {"rich_text": [{"text": {"content": row['문제+보기']}}]},
            "정답": {"number": row['정답']}
        }
```

```
    )

print(f"✅ 960문제 노션 이관 완료")
```

주 1회 자동 배포 (GitHub Actions):

```yaml
# .github/workflows/sync-notion.yml
name: Sync Notion to JSON

on:
  schedule:
    - cron: '0 0 * * 0'  # 매주 일요일 0시
  workflow_dispatch:     # 수동 실행 가능

jobs:
  sync:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Setup Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install notion-client pandas

      - name: Export Notion to JSON
        env:
          NOTION_TOKEN: ${{ secrets.NOTION_TOKEN }}
          NOTION_DB_ID: ${{ secrets.NOTION_DB_ID }}
        run: python scripts/export_notion.py

      - name: Commit and Push
        run: |
          git config user.name "GitHub Actions"
          git config user.email "actions@github.com"
          git add public/data/exams.json
          git commit -m "chore: update exam data from Notion"
          git push
```

export_notion.py:

```python
# scripts/export_notion.py
import json
from notion_client import Client

notion = Client(auth=os.getenv('NOTION_TOKEN'))
db_id = os.getenv('NOTION_DB_ID')

# 노션에서 전체 문제 조회
response = notion.databases.query(
    database_id=db_id,
    filter={"property": "공개여부", "checkbox": {"equals": True}},
    sorts=[
        {"property": "회차", "direction": "ascending"},
        {"property": "문번", "direction": "ascending"}
    ]
)

# JSON 변환
exams = []
for page in response['results']:
    props = page['properties']
    exams.append({
        'id': props['문제ID']['title'][0]['text']['content'],
        'round': props['회차']['number'],
        'number': props['문번']['number'],
        'part': props['과목']['select']['name'],
        'subject': props['세부과목']['select']['name'],
        'question': props['문제전체']['rich_text'][0]['text']['content'],
        'answer': props['정답']['number']
    })

# JSON 파일 저장
with open('public/data/exams.json', 'w', encoding='utf-8') as f:
    json.dump(exams, f, ensure_ascii=False, indent=2)

print(f"✅ {len(exams)}개 문제 JSON 저장 완료")
```

프론트엔드 JSON 로드

```javascript
// src/utils/loadExams.js
export async function loadExams() {
  try {
    // 정적 JSON 파일 로드 (노션 API 미사용)
    const response = await fetch('/data/exams.json');
```

```
    const exams = await response.json();

    // 회차별 그룹핑
    const examsByRound = exams.reduce((acc, exam) => {
      if (!acc[exam.round]) acc[exam.round] = [];
      acc[exam.round].push(exam);
      return acc;
    }, {});

    return examsByRound;
  } catch (error) {
    console.error('❌ 문제 로드 실패:', error);
    // 사용자에게 에러 표시
    return null;
  }
}
```

로컬 스토리지 학습 이력

```javascript
// localStorage 구조
const guestData = {
  deviceId: "uuid-generated",
  createdAt: "2026-02-17T10:00:00Z",

  // 현재 진도
  currentRound: 23,
  currentSubject: "보험업법",
  currentQuestion: 15,

  // 학습 이력
  history: [
    {
      questionId: "23-001-보험업법",
      userAnswer: 2,
      correctAnswer: 2,
      isCorrect: true,
      attemptedAt: "2026-02-17T10:05:00Z",
      timeSpent: 45  // 초
    }
  ],

  // 과목별 진도
  progress: {
    "보험업법": { completed: 10, total: 80 },
    "상법": { completed: 0, total: 160 },
    // ... 12개 과목
```

```
  }
};

localStorage.setItem('gep_guest', JSON.stringify(guestData));
```

Phase 1 제공 기능

- ✅ 23~30회 기출문제 (960문제)
- ✅ 회차별/세부과목별 학습
- ✅ 정답 즉시 확인
- ✅ 진도 저장 (로컬)
- ✅ 익명 사용 (회원가입 불필요)
- ❌ 통계 기능 (Phase 2)
- ❌ 틀린 문제 복습 (Phase 2)
- ❌ 서버 저장 (Phase 2)

Phase 1 완료 기준

- ✅ 960문제 전체 풀이 가능
- ✅ JSON 파일 정상 로드
- ✅ localStorage 저장 작동
- ✅ GitHub Actions 자동화 완료
- ✅ Vercel 배포 성공

Phase 1 비용

$0 (완전 무료)

- 노션: 무료 플랜 (월 1,000 요청 제한 회피)
- Vercel: 무료 플랜 (100GB 대역폭)
- GitHub Actions: 무료 (월 2,000분)
- 도메인: gep-exam.vercel.app (무료)

Phase 2: Supabase 전환 (4주, 무료티어)

목표: 레벨2~3 유료 서비스 검증

기술 스택

- **DB:** Supabase PostgreSQL (무료티어 500MB)
- **인증:** Supabase Auth
- **저장소:** Supabase Storage (이미지 등)
- **실시간:** Supabase Realtime (선택)
- **결제:** 토스페이먼츠
- **호스팅:** Vercel (무료)

- **비용:** 무료티어 (PG 수수료만)

Supabase 테이블 설계 (6개, 간소화)

```sql
-- 1. users (회원)
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  email VARCHAR(255) UNIQUE NOT NULL,
  name VARCHAR(100),
  subscription_level INTEGER DEFAULT 1, -- 1~5
  created_at TIMESTAMP DEFAULT NOW()
);

-- 2. exam_rounds (회차 정보)
CREATE TABLE exam_rounds (
  id SERIAL PRIMARY KEY,
  round_number INTEGER UNIQUE NOT NULL, -- 23~32
  question_count INTEGER DEFAULT 120,
  exam_date DATE,
  status VARCHAR(20) DEFAULT 'active'
);

-- 3. questions (문제, 노션 데이터 복사)
CREATE TABLE questions (
  id VARCHAR(50) PRIMARY KEY, -- "23-001-보험업법"
  round_id INTEGER REFERENCES exam_rounds(id),
  number INTEGER,
  part VARCHAR(10), -- 1부/2부/3부
  subject VARCHAR(50), -- 보험업법, 상법...
  question_text TEXT,
  answer INTEGER -- 1~4
);

-- 4. learning_history (학습 이력)
CREATE TABLE learning_history (
  id SERIAL PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  question_id VARCHAR(50) REFERENCES questions(id),
  user_answer INTEGER,
  is_correct BOOLEAN,
  attempted_at TIMESTAMP DEFAULT NOW(),
  time_spent INTEGER -- 초
);

-- 5. user_stats (통계 요약)
CREATE TABLE user_stats (
  id SERIAL PRIMARY KEY,
```

```
  user_id UUID REFERENCES users(id),
  subject VARCHAR(50),
  total_questions INTEGER DEFAULT 0,
  correct_answers INTEGER DEFAULT 0,
  accuracy DECIMAL(5,2), -- 정답률 %
  last_updated TIMESTAMP DEFAULT NOW(),
  UNIQUE(user_id, subject)
);

-- 6. subscriptions (구독)
CREATE TABLE subscriptions (
  id SERIAL PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  level INTEGER, -- 2~5
  started_at TIMESTAMP DEFAULT NOW(),
  expires_at TIMESTAMP,
  status VARCHAR(20) DEFAULT 'active', -- active/expired/cancelled
  payment_id VARCHAR(100) -- 토스페이먼츠 결제ID
);
```

localStorage → Supabase 마이그레이션

```javascript
// 회원가입 시 자동 마이그레이션
async function signUpAndMigrate(email, password) {
  // 1. 로컬 데이터 수집
  const localData = JSON.parse(localStorage.getItem('gep_guest'));

  // 2. 회원가입
  const { data: { user }, error } = await supabase.auth.signUp({
    email,
    password
  });

  if (error) throw error;

  // 3. 학습 이력 서버 전송
  const historyInserts = localData.history.map(h => ({
    user_id: user.id,
    question_id: h.questionId,
    user_answer: h.userAnswer,
    is_correct: h.isCorrect,
    attempted_at: h.attemptedAt,
    time_spent: h.timeSpent
  }));

  await supabase.from('learning_history').insert(historyInserts);
```

```
  // 4. 통계 재계산
  await recalculateUserStats(user.id);

  // 5. 로컬 데이터 백업 표시
  localStorage.setItem('gep_migrated', 'true');
  localStorage.setItem('gep_backup', JSON.stringify(localData));

  console.log('✅ 마이그레이션 완료');
}
```

Phase 2 제공 기능

- ✅ Phase 1 모든 기능 +
- ✅ 회원가입 / 로그인
- ✅ 서버 데이터 보존
- ✅ **과목별 정답률 통계** (Phase 1 이월)
- ✅ **틀린 문제 자동 모음** (Phase 1 이월)
- ✅ 학습 진도 추적
- ✅ 구독 결제 (레벨2~3)
- ✅ **정답 해설** (Phase 1 이월)
- ✅ **35분 타이머** (Phase 1 이월)

Phase 2 전환 조건 (확정)

- ✅ 가입자 **300명** 이상
- ✅ 활성도 **50%** 이상 (주 1회 접속)
- ✅ 평균 학습 문제 100개 이상
- ✅ 피드백 긍정률 70% 이상

Phase 2 비용

무료티어 + 변동

- Supabase: $0 (무료티어 500MB, 월 5만 요청)
- Vercel: $0
- 토스페이먼츠 PG 수수료: **3.3%**
- 예시: 월 매출 100만원 → 수수료 3.3만원

Phase 3: PostgreSQL 전환 (6주, $145~200/월)

목표: 레벨4~5 본격 서비스

기술 스택

- **DB:** PostgreSQL 14 (AWS RDS)
- **서버:** Node.js + Express (AWS EC2)
- **캐싱:** Redis (AWS ElastiCache)
- **스토리지:** AWS S3 + CloudFront
- **AI:** Claude API (진위형 변환)
- **결제:** 토스페이먼츠
- **모니터링:** CloudWatch
- **비용:** $145~200/월

PostgreSQL 14개 테이블 설계 (1.0 설계 재사용)

```sql
-- 핵심 테이블 (Phase 2와 유사, 확장)
1. users
2. exam_rounds
3. questions
4. learning_history
5. user_stats

-- 추가 테이블 (Phase 3 신규)
6. true_false_questions -- 진위형 6,000문제
7. mock_exams -- 모의고사 패키지
8. mock_exam_results -- 모의고사 결과
9. study_plans -- AI 추천 학습 플랜
10. feedback_logs -- 사용자 피드백
11. payment_logs -- 결제 로그
12. admin_logs -- 관리자 액션 로그
13. notifications -- 푸시 알림
14. referrals -- 추천 프로그램
```

Supabase → PostgreSQL 마이그레이션

```javascript
// 일괄 마이그레이션 스크립트
async function migrateToPostgreSQL() {
  // 1. Supabase 전체 데이터 덤프
  const users = await supabase.from('users').select('*');
  const history = await supabase.from('learning_history').select('*');
  // ... 모든 테이블

  // 2. PostgreSQL 스키마 생성 (Prisma 사용)
  await prisma.$executeRaw`CREATE TABLE IF NOT EXISTS ...`;

  // 3. 데이터 변환 및 삽입
  await prisma.user.createMany({ data: transformUsers(users.data) });
  await prisma.learningHistory.createMany({ data: transformHistory(history.data) });
```

```
  // 4. 통계 재계산
  await recalculateAllStats();

  console.log('✅ PostgreSQL 마이그레이션 완료');
}
```

Phase 3 제공 기능

- ✅ Phase 2 모든 기능 +
- ✅ **진위형 문제 6,000개** (AI 생성)
- ✅ 출처 추적 시스템
- ✅ 틀린 횟수별 필터 (3/5/7회)
- ✅ **모의고사 패키지** (기출 재구성)
- ✅ 합격확률 예측 알고리즘
- ✅ AI 학습 조언 챗봇
- ✅ 관리자 대시보드

Phase 3 전환 조건

- ✅ 유료 가입자 50명 이상
- ✅ 월 매출 100만원 이상
- ✅ 유료 전환율 10% 이상
- ✅ 서버 비용 감당 가능

Phase 3 비용

항목	월 비용	비고
AWS RDS (db.t3.micro)	$50	PostgreSQL
AWS EC2 (t3.micro)	$10	Node.js 서버
ElastiCache (Redis)	$15	캐싱
Claude API	$50	진위형 생성
S3 + CloudFront	$20	정적파일
토스 PG 수수료	3.3%	변동
합계	**$145~200/월**	

---

3. 백엔드 비교표

항목	Phase 1 노션/JSON	Phase 2 Supabase	Phase 3 PostgreSQL
목적	시장 검증	유료 검증	본격 서비스
기간	4주	4주	6주
비용	$0	무료티어	$145~200/월
데이터	노션 DB (원본) JSON (배포)	Supabase PG	AWS RDS PG
테이블 수	0개 (서버 없음)	6개 (간소화)	14개 (풀스택)

학습 이력	localStorage	서버 저장	서버 저장 + 분석
인증	없음	Supabase Auth	JWT + 본인인증
결제	없음	토스페이먼츠	토스페이먼츠
캐싱	없음	없음	Redis
AI 연동	없음	없음	Claude API
제공 레벨	레벨1	레벨1~3	레벨1~5
문제 수	960 (23~30회)	960 + 31~32회	1,200 + 진위형 6,000
호스팅	Vercel	Vercel	AWS EC2
확장성	제한적	중간	높음
개발 난이도	낮음	중간	높음
API 방식	정적 JSON	REST API	REST API + GraphQL

---

4. 단계별 마이그레이션 전략

4.1 Phase 1 → 2 전환

전환 시점:

- 가입자 300명 달성
- 활성도 50% 이상

마이그레이션 절차:

1. **Supabase 프로젝트 생성**

   ```bash
   npx supabase init
   npx supabase start
   ```

1. **스키마 배포**

   ```sql
   -- migrations/001_initial_schema.sql
   CREATE TABLE users (...);
   CREATE TABLE questions (...);
   -- ... 6개 테이블
   ```

1. **노션 데이터 → Supabase 복사**

   ```python
   # 노션 JSON → Supabase 일괄 삽입
   import json
   from supabase import create_client

   with open('public/data/exams.json') as f:
       questions = json.load(f)
   ```

```
supabase.table('questions').insert(questions).execute()
```

1. **프론트엔드 API 전환**

```javascript
// 기존: fetch('/data/exams.json')
// 신규: supabase.from('questions').select('*')
```

1. **localStorage 자동 마이그레이션**
- 회원가입 시 로컬 데이터 서버 전송
- 백업 보관

4.2 Phase 2 → 3 전환

전환 시점:

- 유료 50명 달성
- 월 매출 100만원 이상

마이그레이션 절차:

1. **AWS 인프라 구축**

```bash
# RDS PostgreSQL 생성
# EC2 인스턴스 생성
# ElastiCache Redis 생성
```

1. **병렬 동기화 (무중단 전환)**

```javascript
// 양쪽 DB에 동시 저장
async function recordAnswer(userId, questionId, answer) {
  // Supabase 저장 (기존)
  await supabase.from('learning_history').insert({
    user_id: userId,
    question_id: questionId,
    user_answer: answer
  });

  // PostgreSQL 저장 (신규, 백그라운드)
  try {
    await prisma.learningHistory.create({
      data: { userId, questionId, userAnswer: answer }
    });
  } catch (error) {
    console.error('PostgreSQL 동기화 실패:', error);
```

```
    // 실패해도 사용자 경험 영향 없음
  }
}
```

1. **데이터 검증 후 전환**

```javascript
// 1주일 병렬 운영 후 데이터 일치 확인
// PostgreSQL 우선으로 전환
// Supabase 읽기 전용으로 유지 (백업)
```

4.3 검증 체크리스트

Phase 1 → 2 전환 조건:

- [x] 가입자 300명 이상
- [x] 무료 사용자 활성도 50% 이상
- [x] 평균 학습 문제 100개 이상
- [x] Supabase 무료티어 용량 충분 확인

Phase 2 → 3 전환 조건:

- [ ] 유료 가입자 50명 이상
- [ ] 월 매출 100만원 이상
- [ ] 유료 전환율 10% 이상
- [ ] AWS 비용 $200/월 감당 가능
- [ ] 진위형 생성 수요 확인

---

5. 개발 일정

5.1 Phase 1: 노션/JSON MVP (4주)

주차	작업	담당	산출물
Week 0	**사전 준비 (3일)**		
Day 1~2	엑셀→노션 자동화 스크립트	고팀장	excel_to_notion.py
Day 3	960문제 노션 이관 + 검증	고팀장 + 조대표님	노션 DB 완성
Week 1	**프론트 개발**		
	React + Vite 세팅	고팀장	프로젝트 구조
	QuestionView 컴포넌트	고팀장	문제 풀이 UI
	localStorage 저장 로직	고팀장	진도 저장
Week 2	**데이터 연동**		
	노션→JSON export 스크립트	고팀장	export_notion.py

	GitHub Actions 설정	고팀장	sync-notion.yml
	회차/과목 선택 UI	고팀장	선택 인터페이스
Week 3	**완성 & 배포**		
	PWA 기본 설정	고팀장	manifest.json
	전체 테스트	전체	버그 수정
	Vercel 배포	고팀장	배포 완료
	베타 오픈 (23~25회)	전체	베타 서비스

Phase 1 완료 기준:

- ✅ 레벨1 무료 서비스 오픈
- ✅ 960문제 제공
- ✅ 로컬 진도 저장
- ✅ GitHub Actions 자동화

5.2 Phase 2: Supabase (4주)

주차	작업
1주	Supabase 프로젝트 생성 & 6개 테이블 스키마
1주	노션 JSON → Supabase 마이그레이션
2주	회원가입/로그인 (Supabase Auth)
2주	구독/결제 (토스페이먼츠)
3주	**통계 기능** (Phase 1 이월)
3주	**틀린 문제 복습** (Phase 1 이월)
4주	테스트 & 정식 오픈

Phase 2 완료 기준:

- ✅ 레벨1~3 서비스 오픈
- ✅ 유료 결제 가능
- ✅ 1,200문제 제공 (31~32회 추가)

5.3 Phase 3: PostgreSQL (6주)

주차	작업
1주	AWS 인프라 구축 (RDS, EC2, Redis)
2주	PostgreSQL 14개 테이블 구축
3주	Supabase → PostgreSQL 병렬 동기화
4주	진위형 생성 시스템 (Claude API)
5주	모의고사 패키지 시스템
6주	테스트 & 전체 완성

Phase 3 완료 기준:

- ✅ 레벨1~5 전체 완성
- ✅ 진위형 6,000문제
- ✅ 모의고사 시스템

---

6. 위험 관리

6.1 Phase별 리스크

Phase 1 리스크:

- ~~노션 API Rate Limit (월 1,000 요청)~~ → ✅ JSON 방식으로 회피
- GitHub Actions 장애 → 수동 실행 백업
- Vercel 배포 실패 → Netlify 대체 준비

해결:

- 정적 JSON 배포로 API 미사용
- 수동 실행 가능하도록 스크립트 분리

Phase 2 리스크:

- Supabase 무료티어 초과 (500MB)
- 해결: 텍스트만 저장, 이미지 제외, 압축

Phase 3 리스크:

- 서버 비용 부담
- 해결: 매출 검증 후 투자, 초기 최소 스펙

6.2 롤백 계획

```
Phase 2 실패 시 → Phase 1 유지 (JSON)
Phase 3 실패 시 → Phase 2 유지 (Supabase)
```

각 Phase는 독립적으로 작동 (레고블럭 철학)

---

7. 결론

7.1 핵심 전략

"프론트 우선, 검증 후 투자"

1. **Phase 1 (노션/JSON):** 시장 검증, $0 투자
1. **Phase 2 (Supabase):** 유료 검증, 무료티어
1. **Phase 3 (PostgreSQL):** 본격 서비스, 검증 후 투자

7.2 기존 설계 활용

1.0의 PostgreSQL 14개 테이블 설계는 Phase 3에서 100% 활용

- 레고블럭: 모듈 독립성 유지
- 탑다운: 확장 가능 구조 유지
- 품질 우선: 검증된 설계 재사용

7.3 성공 지표

Phase	목표 지표	다음 단계 조건
Phase 1	베타 사용자 10명 테스트	피드백 수집
Phase 2 전환	가입자 300명, 활성도 50%	수익화 시작
Phase 3 전환	유료 50명, 월 매출 100만원	사업 확장

📌 최종 확인

작성자: 노팀장
승인자: 조대표님
승인일: 2026.02.17
상태: ✅ **개발 착수 승인**

다음 문서:

- GEP-007 프론트엔드 기획안 2.0
- GEP-009 고팀장 인수인계서

작성 완료
노팀장 드림